Numerical Analysis: Midterm (**60 marks**)
Solutions

Urbain Vaes

October 23, 2023

Question 1 (Floating point arithmetic, **12 marks**)**.** True or false? (+1/0/-1)

1. Let $(\bullet)_2$ denote binary representation. It holds that $(0.1111)_2 + (0.0001)_2 = 1$.
2. It holds that $(1000)_{16} \times (0.001)_{16} = 1$.
3. It holds that
$$(0.\overline{1})_3 = \frac{1}{2}.$$
4. In base 16, all the natural numbers from 1 to 300 can be represented using 2 digits.
5. If x is finite `Float64` number, then x is a rational number.
6. In Julia, `eps(2.0)` returns the machine epsilon for the Float64 format.
7. Storing the matrix obtained with the Julia command `zeros(10^5, 10^5)` would require more than 50GB of memory.
8. The spacing (in absolute value) between successive double-precision (`Float64`) floating point numbers is constant.
9. It holds that $(0.\overline{10101})_2 = (1.2345)_{10}$.
10. Machine addition $\widehat{+}$ is an associative operation. More precisely, given any three double-precision floating point numbers x, y and z, the following equality holds:
$$(x \mathbin{\widehat{+}} y) \mathbin{\widehat{+}} z = x \mathbin{\widehat{+}} (y \mathbin{\widehat{+}} z).$$
11. The machine epsilon is the smallest strictly positive number that can be represented in a floating point format.
12. In Julia, the command `nextfloat(2.)` returns the next `Float32` number after 2.

Solution.

1. **True**: $(0.1111)_2 + (0.0001)_2 = \frac{15}{16} + \frac{1}{16} = 1$.
2. **True**: $(1000)_{16} = 16^3$ and $(0.001)_{16} = 16^{-3}$.
3. **True**: $(0.\overline{1})_3 = \sum_{k=1}^{\infty} 3^{-k} = \frac{1/3}{1-1/3} = \frac{1}{2}$.
4. **False**: with 2 hexadecimal digits, only the numbers up to $(\mathrm{FF})_{16} = 255$ can be represented.
5. **True**: a finite `Float64` number is of the form $\pm m \times 2^e$ with m, e integers, which is rational.
6. **False**: `eps(2.0)` returns the spacing between 2 and the next `Float64` number, which is 2^{-51}, twice the machine epsilon `eps()` $= 2^{-52}$.
7. **True**: the matrix contains 10^{10} numbers of 8 bytes each, i.e. 8×10^{10} bytes = 80GB.
8. **False**: the spacing between successive numbers in $[2^e, 2^{e+1})$ is 2^{e-52}, which depends on e.
9. **False**: the left-hand side is less than 1, whereas the right-hand side is greater than 1.
10. **False**: for example, with $\varepsilon = 2^{-52}$, $(1 \widehat{+} \varepsilon/2) \widehat{+} \varepsilon/2 = 1$, whereas $1 \widehat{+} (\varepsilon/2 \widehat{+} \varepsilon/2) = 1 + \varepsilon$.
11. **False**: the machine epsilon is the distance between 1 and the next floating point number. Much smaller numbers are representable, e.g. `floatmin()` $= 2^{-1022}$ or the subnormal `nextfloat(0.0)`.
12. **False**: `2.` is a `Float64` number, so `nextfloat(2.)` returns the next `Float64` number, $2 + 2^{-51}$.

△

Question 2 (Interpolation and approximation, **10 marks**)**.** Throughout this exercise, we use the notation $x_i^n = i/n$ and assume that $u \colon \mathbf{R} \to \mathbf{R}$ is a continuous function. The notation $\mathbf{P}(n)$ denotes the set of polynomials of degree less than or equal to n. We proved in class that, for all $n \geqslant 0$, there exists a unique polynomial $p_n \in \mathbf{P}(n)$ such that

$$\forall i \in \{0, \dots, n\}, \qquad p_n(x_i^n) = u(x_i^n). \tag{1}$$

1. The degree of p_n is exactly n or $n-1$.
2. Suppose that $u \in \mathbf{P}(m)$. Then $p_n = u$ if $n \geqslant m$.
3. Fix $u(x) = \sin(3\pi x)$. Then $p_3(x) = 0$.
4. Fix $u(x) = \cos(\pi x)$. Then $p_2(x) = (2x-1)^2$.
5. For all u that is smooth, it holds that

$$\max_{x \in [0,1]} \left| u(x) - p_n(x) \right| \xrightarrow[n \to \infty]{} 0.$$

6. Fix $u(x) = \cos(2x)$. Then

$$\max_{x\in[0,1]} \big|u(x) - p_n(x)\big| \xrightarrow[n\to\infty]{} 0.$$

7. Fix $u(x) = \sin(x)$. Then

$$\max_{x\in\mathbf{R}} \big|u(x) - p_n(x)\big| \xrightarrow[n\to\infty]{} 0.$$

8. Suppose that $p(x) \in \mathbf{P}(n)$ and let $q(x) = p(x+1) - p(x)$. Then $q \in \mathbf{P}(n-1)$.

9. Let $(f_0, f_1, f_2, \dots) = (1, 1, 2, \dots)$ denote the Fibonacci sequence. There exists a polynomial p such that

$$\forall n \in \mathbf{N}, \qquad f_n = p(n).$$

10. For any matrix $\mathsf{A} \in \mathbf{R}^{20\times 10}$, the linear system

$$\mathsf{A}^T\mathsf{A}\boldsymbol{\alpha} = \mathsf{A}^T\boldsymbol{\alpha}$$

admits a unique solution.

Solution.

1. **False**: for example, if u is constant, then p_n is constant for all n.
2. **True**: if $n \geqslant m$, then $u \in \mathbf{P}(n)$ satisfies the interpolation conditions, so $p_n = u$ by uniqueness.
3. **True**: $u(x_i^3) = \sin(\pi i) = 0$ for $i \in \{0, 1, 2, 3\}$.
4. **False**: $u(0) = 1$, $u(\frac{1}{2}) = 0$ and $u(1) = -1$, so $p_2(x) = 1 - 2x$. The polynomial $(2x-1)^2$ takes the value 1 at $x = 1$.
5. **False**: for equidistant nodes, the interpolant does not converge for all smooth functions. A counterexample is a Runge function such as $u(x) = \big(1 + 100(x - \frac{1}{2})^2\big)^{-1}$.
6. **True**: $|u^{(n+1)}(x)| \leqslant 2^{n+1}$, so the interpolation error formula gives

$$\max_{x\in[0,1]} \big|u(x) - p_n(x)\big| \leqslant \frac{2^{n+1}}{(n+1)!} \max_{x\in[0,1]} \prod_{i=0}^{n} |x - x_i^n| \leqslant \frac{2^{n+1}}{(n+1)!} \xrightarrow[n\to\infty]{} 0.$$

7. **False**: for $n \geqslant 1$, the polynomial p_n is not constant, because u is not constant on $\{x_0^n, \dots, x_n^n\}$, so p_n is unbounded on $\mathbf{R}$, while u is bounded. Therefore the maximum is infinite.
8. **True**: the terms of degree n in $p(x+1)$ and $p(x)$ cancel.

9. **False**: the Fibonacci numbers grow exponentially, like ϕ^n with $\phi = \frac{1+\sqrt{5}}{2}$, faster than any polynomial. (Alternatively, $\Delta f_n = f_{n+1} - f_n = f_{n-1}$, so no iterated difference of the sequence vanishes.)

10. **False**: (the right-hand side should read $\mathsf{A}^T \boldsymbol{b}$.) The normal equations have a unique solution if and only if $\mathsf{A}^T\mathsf{A}$ is invertible, i.e. if the columns of A are linearly independent. This is not the case for every matrix, e.g. $\mathsf{A} = 0$.

△

Question 3 (Numerical integration, **9 marks**)**.** True or false? (+1/0/-1)

1. The degree of precision of the following rule is equal to 2:

$$\int_{-1}^{1} u(x)\,\mathrm{d}x \approx 2u(0).$$

2. The degree of precision of the following rule is equal to 3:

$$\int_{-1}^{1} u(x)\,\mathrm{d}x \approx u\left(-\frac{1}{3}\right) + u\left(\frac{1}{3}\right).$$

3. For any natural number $N > 0$, there exists a quadrature rule with a degree of precision equal to $2N + 1$ of the form

$$\int_{-1}^{1} u(x)\,\mathrm{d}x \approx \sum_{n=0}^{N} w_n u(x_n).$$

4. Legendre polynomials are orthogonal for the inner product

$$\langle f, g\rangle := \int_{-1}^{1} f(x)\,g(x)\,\mathrm{d}x.$$

5. Fix $u(x) = \cos(x)$ and let

$$I_N^{MC} = \frac{1}{N}\sum_{n=1}^{N} u(X_n), \qquad X_n \overset{\text{i.i.d.}}{\sim} \mathcal{U}([0,1]). \tag{2}$$

   The expectation of $\widehat{I}_N$ is independent of N.

6. The variance of I_N^{MC} in (2) tends to 0 in the limit $N \to \infty$.

7. Let $x_i^N = i/N$ and consider the following approximation of $\int_0^1 u(x)\,\mathrm{d}x$:

$$I_N^T = \frac{1}{2N}\Big(u\left(x_0^N\right) + 2u\left(x_1^N\right) + 2u\left(x_2^N\right) + \ldots + u\left(x_{N-2}^N\right) + 2u\left(x_{N-1}^N\right) + u\left(x_N^N\right)\Big). \tag{3}$$

Fix $u(x) = (1 + 25x^2)^{-1}$. Then I_N^T diverges in the limit $N \to \infty$.

8. Fix $u(x) = \cos(x)$ and let I_N^T be as in (3). Then there exists $C \in (0, \infty)$ such that

$$\forall N \geqslant 2, \qquad \left| I_N^T - \int_0^1 u(x)\,\mathrm{d}x \right| \leqslant \frac{C}{N}.$$

9. Fix $u(x) = 2x - 1$ and let I_N^T be as in (3). Then $I_N^T = 0$ for $N \geqslant 2$.

Solution.

1. **False**: the rule is exact for 1 and x but not for x^2, so its degree of precision is 1.
2. **False**: the rule is exact for 1, and for x and x^3 by symmetry, but not for x^2: it gives $\frac{2}{9}$ instead of $\frac{2}{3}$. Its degree of precision is 1. (The Gauss–Legendre rule with 2 nodes, which has degree of precision 3, uses the nodes $\pm\frac{1}{\sqrt{3}}$.)
3. **True**: the Gauss–Legendre rule with $N+1$ nodes has degree of precision $2(N+1)-1 = 2N+1$.
4. **True**: this is the defining property of Legendre polynomials.
5. **True**: by linearity of the expectation, $\mathbf{E}\left[I_N^{MC}\right] = \mathbf{E}\left[u(X_1)\right] = \int_0^1 \cos(x)\,\mathrm{d}x = \sin(1)$.
6. **True**: since the X_n are independent, $\mathbf{V}\left[I_N^{MC}\right] = \frac{1}{N}\mathbf{V}\left[u(X_1)\right] \to 0$.
7. **False**: (the coefficient of $u(x_{N-2}^N)$ in (3) should be 2.) The composite trapezium rule converges for any continuous function, with an error of order N^{-2} for a smooth function such as this one. The Runge phenomenon concerns polynomial interpolation on equidistant nodes, not the composite trapezium rule.
8. **True**: the error is bounded by $\frac{1}{12N^2}\max_{[0,1]}|u''| \leqslant \frac{1}{12N}$.
9. **True**: the trapezium rule is exact for linear functions, and $\int_0^1 (2x-1)\,\mathrm{d}x = 0$.

△

💻 **Computer exercise 1** (Floating point arithmetic, **10 marks**)**.** Read the documentation of the `nextfloat` function. Using this function, plot on the same graph the spacing between successive `Float16`, `Float32` and `Float64` numbers in the range $[1, 10^4]$. Use a logarithmic scale for the x and y axes. You may find it useful to use `LinRange{Type}(a, b, n)` to create a vector of n equidistant numbers of type `Type` between a and b.

Solution. The spacing between a number `x` and the next floating point number of the same type is `nextfloat(x) - x`:

```
using Plots
spacing(x) = nextfloat(x) - x
plot(xaxis=:log, yaxis=:log, xlabel="x", ylabel="Spacing", legend=:bottomright)
```

```
for T in (Float16, Float32, Float64)
    x = LinRange{T}(1, 10^4, 1000)
    plot!(x, spacing.(x), label=string(T))
end
```

The spacing is piecewise constant, and it doubles at each power of 2: for $x \in [2^e, 2^{e+1})$, it is equal to $2^e \times$ `eps(T)`. In logarithmic scale, the three curves are therefore staircases that lie approximately on the lines $y = x \times$ `eps(T)`. For example, at $x = 10^4 \in [2^{13}, 2^{14})$, the spacing is $2^{13-10} = 8$ for `Float16`, $2^{13-23} \approx 9.8 \times 10^{-4}$ for `Float32` and $2^{13-52} \approx 1.8 \times 10^{-12}$ for `Float64`. △

Computer exercise 2 (Interpolation, **10 marks**)**.** Consider the data

$$x = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}, \qquad y = \begin{pmatrix} 12 \\ 7 \\ 2 \\ 4 \end{pmatrix}.$$

Find $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ such that

$$\widehat{u}(x) := \alpha_1 \cos(\pi x) + \frac{\alpha_2}{x} + \alpha_3 \, 2^x + \alpha_4 \, x^2.$$

satisfies

$$\forall i \in \{1, 2, 3, 4\}, \qquad \widehat{u}(x_i) = y_i.$$

Plot on the same graph the function $\widehat{u}$ and the data points.

Solution. The interpolation conditions form a linear system of size 4 for the coefficients:

```
using Plots
x = [1, 2, 3, 4]
y = [12, 7, 2, 4]
A = [cos.(pi*x) 1 ./ x 2.0 .^ x x.^2]
alpha = A \ y  # [1.0, 12.0, 1.0, -1.0]

u(t) = alpha[1] * cos(pi*t) + alpha[2] / t + alpha[3] * 2^t + alpha[4] * t^2
plot(u, 1, 4, label="Interpolant")
scatter!(x, y, label="Data")
```

We obtain $\alpha_1 = 1$, $\alpha_2 = 12$, $\alpha_3 = 1$ and $\alpha_4 = -1$, that is $\widehat{u}(x) = \cos(\pi x) + \frac{12}{x} + 2^x - x^2$. One can check that $\widehat{u}(1) = -1 + 12 + 2 - 1 = 12$, $\widehat{u}(2) = 1 + 6 + 4 - 4 = 7$, and so on. △

Computer exercise 3 (Approximation, **10 marks**)**.** Write, without using the `Polynomials`

library, a function `approx(x, y, d, X)` to obtain, given data points

$$(x_0, y_0), \ldots, (x_N, y_N)$$

and a nonnegative integer $0 \leqslant d \leqslant N$, the polynomial $p \in \mathbf{P}(d)$ minimizing the total error

$$E := \frac{1}{2} \sum_{n=0}^{N} \left| p(x_n) - y_n \right|^2.$$

Your function should a vector containing the values $p(X_0), \ldots, p(X_M)$, where $X_0, \ldots, X_M$ are the elements of the vector `X`. Within the function, proceed in 3 steps:

- First create the following matrix and vector:

$$\mathsf{A} \begin{pmatrix} 1 & x_0 & \ldots & x_0^d \\ \vdots & \vdots & & \vdots \\ 1 & x_N & \ldots & x_N^d \end{pmatrix}, \qquad \boldsymbol{b} := \begin{pmatrix} y_0 \\ \vdots \\ y_N \end{pmatrix}.$$

- Then solve the normal equations using the backslash operator:

$$\mathsf{A}^T \mathsf{A} \boldsymbol{\alpha} = \mathsf{A}^T \boldsymbol{b}.$$

- Finally, evaluate the polynomial

$$p(x) = \alpha_0 + \alpha_1 x + \ldots + \alpha_d x^d.$$

  at all the points in `X` and return the result in a vector.

Use the data given in the notebook, of the altitude of a marble in free fall as a function of time, to test your code. The experiment was performed on a different planet. Can you find which one? See https://en.wikipedia.org/wiki/Gravitational_acceleration.

Solution.

```
function approx(x, y, d, X)
    A = [xn^j for xn in x, j in 0:d]
    b = y
    alpha = (A' * A) \ (A' * b)
    return [sum(alpha[j+1] * Xm^j for j in 0:d) for Xm in X]
end
```

For the data of the notebook,

```
using Plots
```

```
x = [0., 1., 2., 3., 4., 5.]
y = [100., 98.26, 93.56, 81.79, 71.25, 53.22]
X = LinRange(0, 5, 200)
plot(X, approx(x, y, 2, X), label="Approximation")
scatter!(x, y, label="Data")
```

The altitude of a body in free fall is $h(t) = h_0 + v_0 t - \frac{g}{2}t^2$, so we use $d = 2$. The least squares approximation is $p(t) \approx 100.11 + 0.024\,t - 1.872\,t^2$, which gives $g \approx 2 \times 1.872 \approx 3.74\,\mathrm{m/s^2}$. This corresponds to the gravitational acceleration on Mars ($3.72\,\mathrm{m/s^2}$), or Mercury ($3.70\,\mathrm{m/s^2}$). △

💻 **Computer exercise 4** (Numerical integration, **10 marks**)**.** Milne's integration rule reads

$$\int_{-1}^{1} u(x)\,\mathrm{d}x \approx \frac{2}{3}\left(2f\left(-\frac{1}{2}\right) - f(0) + 2f\left(\frac{1}{2}\right)\right)$$

- Write a function `composite_milne(u, a, b, N)`, which returns an approximation of the integral

$$\int_a^b u(x)\,\mathrm{d}x$$

obtained by partitioning the integration interval $[a, b]$ into N cells, and applying Milne's rule within each cell.

- Take $u(x) = \cos(x)$, $a = -1$ and $b = 1$. Plot the evolution of the error for N varying from 1 to 1000.

- Estimate the order of convergence with respect to N, i.e. find α such that

$$|\widehat{I}_N - I| \propto CN^{-\alpha},$$

where I denotes the exact value of the integral and $\widehat{I}_N$ denotes its approximation. In order to find α, use the function `fit` from the `Polynomials` package to find a linear approximation of the form

$$\log|\widehat{I}_N - I| \approx \log(C) - \alpha \log(N).$$

Solution. • On a cell $[a, b]$, with $h = \frac{b-a}{4}$, Milne's rule reads $\frac{b-a}{3}\big(2u(a+h) - u(a+2h) + 2u(a+3h)\big)$:

```
function milne(u, a, b)
    h = (b - a) / 4
    return (b - a) / 3 * (2u(a + h) - u(a + 2h) + 2u(a + 3h))
end
```

```
function composite_milne(u, a, b, N)
    x = range(a, b, length = N + 1)
    return sum(milne(u, x[i], x[i+1]) for i in 1:N)
end
```

- The exact value is $I = 2\sin(1)$:

  ```
  using Plots
  Ns = 1:1000
  errors = [abs(composite_milne(cos, -1, 1, N) - 2sin(1)) for N in Ns]
  plot(Ns, errors, xaxis=:log, yaxis=:log, label="Error", xlabel="N")
  ```

  The error decreases from about 9×10^{-3} for $N = 1$ to about 8×10^{-11} for $N = 100$ and 6×10^{-15} for $N = 1000$, close to the level of round-off errors.

- ```
  using Polynomials
  p = fit(log.(Ns[1:100]), log.(errors[1:100]), 1)
  alpha = -p[1]  # approximately 4
  ```

  We obtain $\alpha \approx 4$. This is consistent with theory: Milne's rule has degree of precision 3 (it is exact for 1, x^2 and odd functions, but not for x^4), so the error is $\mathcal{O}(h^5)$ in each cell of size $h = \frac{b-a}{N}$, and the total error is $\mathcal{O}(Nh^5) = \mathcal{O}(N^{-4})$.

△
  ```